Mockup

Full Name: _____

30 minutes, one handwritten or 4x6 index card (or A6) cheat sheet allowed, no printed material.

---

1. Two algorithms A and B perform the same task. Algorithm A has a worst-case complexity of O(NlogN). Algorithm B has a worst-case complexity of O(N). Both are executed on the same input of size N = 1000, on the same computer: A takes 1 ms, B takes 3 ms.
Which of the following are true (select one or more):

- ☐ the measurements must be wrong, since B should be faster
- ☐ algorithm A is faster than algorithm B for every N
- ☒ algorithm B is faster than algorithm A for large enough N
- ☐ algorithm A is faster than algorithm B for every $N < 1000$
- ☐ algorithm A is also faster than algorithm B on an input of size $N = 2000$

---

2. The array `{4, 7, 2, 9, 1, 5}` is sorted in ascending order.
Selection sort: each pass finds the smallest element of the unsorted part and swaps it with the first element of the unsorted part. What does the array look like after the first two passes?
Answer: _____ ({1, 2, 7, 9, 4, 5})
Insertion sort: what does the array look like after the first three passes (that is, after the elements at indices 1, 2 and 3 have been inserted)?
Answer: _____ ({2, 4, 7, 9, 1, 5})

---

3. Consider the following class.

```
class Word implements Comparable<Word> {
    String text;
    Word(String text) { this.text = text; }

    public int compareTo(Word other) {
        return text.length() - other.text.length();
    }
}
```

The array below is sorted with a **stable** sorting algorithm, using `compareTo` to compare elements.

```
Word[] words = { new Word("pear"), new Word("fig"), new Word("plum"),
                 new Word("banana"), new Word("kiwi"), new Word("apple"),
                 new Word("date") };
```

In which order are the words after sorting?
Answer: _____ (fig, pear, plum, kiwi, date, apple, banana)

---

4. A sorting algorithm is **stable** if (select one):

- ☐ its running time is the same on every input of size N
- ☐ it runs faster on input that is already partly sorted
- ☒ equal elements keep their original relative order
- ☐ it uses only O(1) extra memory
- ☐ it never compares the same pair of elements twice

---

5. The following method of an array-based list of `double`s should insert `e` at position `pos`, moving the elements from `pos` to `size - 1` up by one position. `data` is the array holding the elements and `size` is the number of elements in the list.

```
void add(int pos, double e) {
    if (pos < 0 || pos > size) throw new IndexOutOfBoundsException();
    if (size == data.length) grow();
    for (int i = pos; i < size; i++)
        data[i + 1] = data[i];
    data[pos] = e;
    size++;
}
```

The list contains `[1, 2, 3, 4]`. What does it contain after `add(0, 9)`?
Answer: _____ ([9, 1, 1, 1, 1])
Fix the method by rewriting the loop.

```
    for (int i = size; i > pos; i--)
        data[i] = data[i - 1];
```

---

6. The following method merges two arrays sorted in ascending order into a new sorted array. Two lines are missing: which of the following completes the method correctly?

```
int[] merge(int[] x, int[] y) {
    int[] result = new int[x.length + y.length];
    int i = 0, j = 0, k = 0;
    while (i < x.length && j < y.length) {
        if (x[i] <= y[j]) result[k++] = x[i++];
        else result[k++] = y[j++];
    }
    ---
    ---
    return result;
}
```

Answer:

☐
```
if (i < x.length) result[k++] = x[i++];
if (j < y.length) result[k++] = y[j++];
```

☒
```
while (i < x.length) result[k++] = x[i++];
while (j < y.length) result[k++] = y[j++];
```

☐
```
while (i < x.length) result[k++] = y[j++];
while (j < y.length) result[k++] = x[i++];
```

☐
```
while (k < result.length) result[k++] = x[i++];
while (k < result.length) result[k++] = y[j++];
```

☐ none of the above, the remaining elements need to be merged with another loop like the one above

---

7. For each statement, mark whether it is part of the List ADT (true for every implementation of a list) or specific to the array-based implementation.

Statement	List ADT	Array-based
Every element has a position in the sequence	☒	☐
After `add(pos, e)`, the elements previously at position pos and after it are one position higher	☒	☐
`get(pos)` takes constant time	☐	☒
The list has a capacity, which can be larger than its size	☐	☒
The elements are stored in consecutive memory cells	☐	☒

---

8. An array-based list doubles the capacity of its array whenever it appends to a full array. Mark each statement as true or false.

Statement	True	False
A single call to `add(e)` on a list of N elements can take time proportional to N	☒	☐
Every call to `add(e)` takes O(1) time	☐	☒
Any sequence of M calls to `add(e)`, starting from an empty list, takes O(M) time in total	☒	☐
O(1) amortized means O(1) on average, over random inputs	☐	☒
Starting from an empty list of capacity 1, 1000 calls to `add(e)` copy fewer than 2000 elements in total	☒	☐

---

9. What does the following code fragment print?

```
List<String> words = new ArrayList<>(List.of("red", "green", "blue"));
Iterator<String> a = words.iterator();
a.next();
a.next();
Iterator<String> b = words.iterator();
System.out.print(b.next() + " " + a.next() + " ");
System.out.print(a.hasNext());
```

Answer: _____ (red blue false)

---

10. (2 points) The array `{5, 8, 3, 9, 2, 7, 1, 6}` is sorted with quicksort, using the **first** element of each subarray as the pivot. Each partition puts every element smaller than the pivot to its left and every larger element to its right, preserving the relative order of the elements that end up on each side. Write the contents of the array after each partition.

```
pivot 5:  [3 2 1 5 8 9 7 6]
pivot 3:  [2 1 3 5 8 9 7 6]
pivot 2:  [1 2 3 5 8 9 7 6]
pivot 8:  [1 2 3 5 7 6 8 9]
pivot 7:  [1 2 3 5 6 7 8 9]
```
